

But this code cannot be compiled with a standard compiler. So let us see how to work with Standard C.

Console operations in Standard C

The problem with the above code is that it is not compatible with Standard C. No standard libraries provide `conio.h`. Using the `void` type for the main function itself is an error. The `int` type may be used and some value returned, normally zero (*return 0*).

Console handling in Standard C is done using *escape sequences*. Escape sequences use `\n` for a newline, `\t` for a horizontal tab, and so on. In Standard C, there are escape sequences that can do many operations on the console. First, let's use these escape sequences. Since they look rough, we will write our own code for the functions `clrscr`, `getch`, `gotoxy`, `textcolor`, `textbackground`, etc.

Some important escape sequences are given in Table 1. You can use them inside `printf` as you use `\n` and `\t`.

Escape code	Use
<code>\x1b[2J</code>	Clears the console.
<code>\x1b;n;mH</code> or <code>\x1b;n;mI</code>	Moves the cursor to row <i>n</i> , column <i>m</i> . The values are 1-based, and default to 1 (top left corner) if omitted.
<code>\x1b?25l</code>	Hides the cursor (<i>Note</i> : the last character is lowercase 'L').
<code>\x1b?25h</code>	Shows the cursor.
<code>\x1b[k;m</code>	Where <i>k</i> is the colour (text colours are from 30 to 37 and background colours are from 40 to 47). For the colour codes, see Table 2.
<code>\x1b[5m</code>	Blinks slowly.
<code>\x1b[6m</code>	Blinks rapidly.

Table 1: Some important ANSI escape codes

As shown in Table 1, you may use `printf("\x1b[2J");` to clear the console, instead of `clrscr()`. You may use `printf("\x1b[10;20H");` instead of `gotoxy(20,10)`; or `printf("\x1b[%d;%dH", a, b)`; instead of `gotoxy(a, b)`.

But while using colours, with `\x1b[k;m`, you have to replace *k* with 30+colour_code in the case of foreground colouring and 40+colour_code in the case of background colouring. Refer to Table 2 for the colour codes.

0	1	2	3	4	5	6	7
Black	Red	Green	Yellow	Blue	Magenta	Cyan	White

Table 2: Colour codes

From Table 2, you can use `printf("\x1b[31m");` instead of `textcolor(RED)`; and `printf("\x1b[41m");` instead of `textbackground(RED)`. You can cancel the colouring using `\x1b[0m`.

To learn more about ANSI escape codes, visit: http://www.en.wikipedia.org/wiki/ANSI_escape_code

These escape codes are used in other languages too, like in Python, when using the `print()` function; e.g.: `print("\x1b[32m")` for a green foreground.

`\x1b[32m")` for a green foreground.

Typical Turbo C programmers start their programs with `clrscr()` to clear the screen and end with `getch()` to wait for a keystroke before the program terminates. Both are meaningless in most cases. If you are familiar with CLI, you might have noticed the irritation while the console clears with every command that you give. A Turbo C programmer gives the `getch()` command. Otherwise the program will disappear suddenly and the result of the program will not be visible. In a standard console, the result stays there for a long time without `getch()`. The extra keystroke is an additional irritation.

However, these two functions are useful in some cases. We have replaced `clrscr()` with an escape code, `\x1b[2J`. But there is no such code for `getch()` since it is a function with return type `char`.

Actually, `getch()` can be replaced with `getchar()` from `stdio.h`. The problem, however, is that `getchar()` waits for the user to press the 'Enter' key. It echoes what was typed, too. We'd like to write our own function that waits only for a single keystroke, which will not echo the keystroke. Here we need the help of an external CLI program, `stty`, which is shipped with almost all GNU/Linux distros.

The code given below shows the combination of escape codes and `stty` to create a program that clears the screen, prints 'Hello, World!' in white with a blue background, waits for a keystroke and resets the video.

Code 2:

```
#include <stdio.h>
#include <stdlib.h>
void clrscr()
{
    printf("\x1b[2J");
}
char getch()
{
    char c; // This function should return the keystroke
    system("stty raw"); // Raw input - wait for only a
    single keystroke
    system("stty -echo"); // Echo off
    c = getchar();
    system("stty cooked"); // Cooked input - reset
    system("stty echo"); // Echo on - Reset
    return c;
}
int main()
{
    printf("\x1b[34m"); // Blue background
    clrscr(); // Clear the screen with blue bg
    printf("\x1b[47m"); // White foreground
    printf("Hello, World!\n");
    getch();
    printf("\x1b[0m"); // Reset the console
```